Experiment 5: Matrix Multiplication Using MapReduce (Windows)

Aim

To implement a Java MapReduce program running on a Windows environment to perform

distributed Matrix Multiplication for large-scale datasets. Procedure
1. Set up the environment variables on your Windows machine to ensure the compiler can reach your

core Java development kit and Hadoop reference libraries. 2. Formulate the matrix file mapping system layout, formatting lines inside the input documents as
comma-separated values matching MatrixName,RowIndex,ColumnIndex,Value tokens. 3. Design the Mapper class logic to project components of Matrix \(A\) and Matrix \(B\) across
matching output coordinates based on rows (\(i\)), shared columns (\(j\)), and columns (\(k\)). 4. Design the Reducer class logic to read the matching combined element streams, sort the matrix

origins using internal hash collection maps, multiply values sharing a standard \(j\) index, and sum

the results. 5. Create the Driver configuration setup to declare input/output key data type definitions (Text) and
specify target input paths within HDFS. 6. Compile your Java files via the Windows Command Prompt using a semicolon-delimited (;)
classpath string pointing to the local %HADOOP_HOME%\share archive directory paths. 7. Wrap your class output binaries inside an executable.jar file and trigger job execution over

sample data logs using the hadoop jar runtime engine. Main Program (MatrixMultiplication.java)

java

import java.io.IOException;import java.util.HashMap;import org.apache.hadoop.conf.Configuration;import org.apache.hadoop.fs.Path;import org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Job;import org.apache.hadoop.mapreduce.Mapper;import org.apache.hadoop.mapreduce.Reducer;import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MatrixMultiplication {

public static class MatrixMapper extends Mapper<Object, Text, Text, Text> {
// Dimensions for target matrices: A is (m x n), B is (n x p)
private int m = 2;
private int p = 2;

public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
String line = value.toString();

String[] tokens = line.split(",");

if (tokens[0].trim().equals("A")) {
String i = tokens[1].trim();
String j = tokens[2].trim();
String valA = tokens[3].trim();
// Send value to all cells in result matrix row i
for (int k = 0; k < p; k++) {
context.write(new Text(i + "," + k), new Text("A," + j + "," + valA));
}
} else if (tokens[0].trim().equals("B")) {
String j = tokens[1].trim();
String k = tokens[2].trim();
String valB = tokens[3].trim();
// Send value to all cells in result matrix column k
for (int i = 0; i < m; i++) {
context.write(new Text(i + "," + k), new Text("B," + j + "," +
valB));
}
}
}
}

public static class MatrixReducer extends Reducer<Text, Text, Text, Text> { private int n = 2; // Dimension common to both matrices

public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
HashMap<Integer, Double> hashA = new HashMap<>();
HashMap<Integer, Double> hashB = new HashMap<>();

for (Text val: values) {
String[] tokens = val.toString().split(",");

if (tokens[0].equals("A")) {
hashA.put(Integer.parseInt(tokens[1]), Double.parseDouble(tokens[2]));
} else if (tokens[0].equals("B")) { hashB.put(Integer.parseInt(tokens[1]), Double.parseDouble(tokens[2]));
}
}

double sum = 0.0;
for (int j = 0; j < n; j++) {
double a = hashA.getOrDefault(j, 0.0);
double b = hashB.getOrDefault(j, 0.0);
sum += a * b;
}

context.write(key, new Text(Double.toString(sum)));
}
}

public static void main(String[] args) throws Exception { Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "matrix multiplication"); job.setJarByClass(MatrixMultiplication.class); job.setMapperClass(MatrixMapper.class); job.setReducerClass(MatrixReducer.class); job.setOutputKeyClass(Text.class); job.setOutputValueClass(Text.class); FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1])); System.exit(job.waitForCompletion(true)? 0: 1);
}
}

Use code with caution.

Windows Execution Commands

cmd

:: 1. Compile the matrix Java source program using Windows library notation links

javac -classpath
"%HADOOP_HOME%\share\hadoop\common*;%HADOOP_HOME%\share\hadoop\common\lib*;%HADOOP_H OME%\share\hadoop\mapreduce*" -d wc_classes MatrixMultiplication.java

:: 2. Compress classes to a JAR archive structure

jar -cvf matrix.jar -C wc_classes/.

:: 3. Prepare an input directory folder structure within the storage cluster

hdfs dfs -mkdir -p /matrix_input

:: 4. Push local test matrix row lines text content upward into HDFS

:: Format: MatrixName,Row,Col,Value

:: Matrix A = [[1,2],[3,4]] | Matrix B = [[5,6],[7,8]]

(echo A,0,0,1 & echo A,0,1,2 & echo A,1,0,3 & echo A,1,1,4 & echo B,0,0,5 & echo B,0,1,6 & echo B,1,0,7 & echo B,1,1,8) > C:\Users\Public\matrix_data.txt

hdfs dfs -put C:\Users\Public\matrix_data.txt /matrix_input/

:: 5. Execute the compiled MapReduce package job across your system nodes

hadoop jar matrix.jar MatrixMultiplication /matrix_input/matrix_data.txt /matrix_output

:: 6. Check output records printed within the resultant partition log dumps

hdfs dfs -cat /matrix_output/part-r-00000

Use code with caution. Execution & Output

text

C:\hadoop\sbin>hdfs dfs -cat /matrix_output/part-r-00000

0,0 19.0

0,1 22.0